

Casa Monarca - Informe de Solución

1. Resumen de la solución

Se desarrolló una aplicación Django completa para la gestión de identidad y acceso de Casa Monarca. La solución incluye:

- Autenticación con Django local (`username` + `password`).
- Soporte opcional de MFA con TOTP (`pyotp`).
- Autorización basada en niveles, áreas y roles.
- Gestión de colaboradores con creación, edición, revocación, reactivación y eliminación lógica.
- Auditoría de acciones administrativas y de inicio de sesión.
- Conexión a MySQL usando `mysql.connector.django`.

2. Esqueleto de la solución

2.1 Estructura de carpetas principal

```
CriptoReto/
├── casamonarca/                                # Proyecto Django
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── iam/                                         # App IAM
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── management/
│   │   └── commands/
│   │       ├── __init__.py
│   │       └── seed_data.py
│   └── migrations/
```

```

├── models.py
├── templates/
│   ├── iam/
│   ├── urls.py
│   ├── views.py
│   └── __init__.py
├── templates/                                # Plantillas globales
│   ├── base.html
│   └── iam/
│       ├── login.html
│       ├── dashboard.html
│       ├── collaborator_form.html
│       ├── collaborator_detail.html
│       └── audit_log.html
├── mysql_setup.sql                          # Script para crear BD y usuario
MySQL
├── requirements.txt
├── README.md
├── manage.py
├── docs/
│   ├── SOLUTION_REPORT.md
│   ├── ARCHITECTURE.mmd
│   └── SQL_SCHEMA.mmd

```

2.2 Componentes clave

- ``casamonarca/settings.py``
 - Configura MySQL mediante variables de entorno.
 - Define ``AUTH_USER_MODEL = 'iam.Collaborator``.

- ``iam/models.py``
 - ``Collaborator``: Modelo de usuario extendido.
 - ``Area``: Tabla de áreas de la organización.
 - ``AuditLog``: Registro de acciones administrativas.
 - ``LoginAttempt``: Control de intentos de acceso.

- ``iam/views.py``
 - ``login_view``, ``logout_view``, ``dashboard_view``.

- CRUD completo de colaboradores.

- Auditoría y control de permisos.

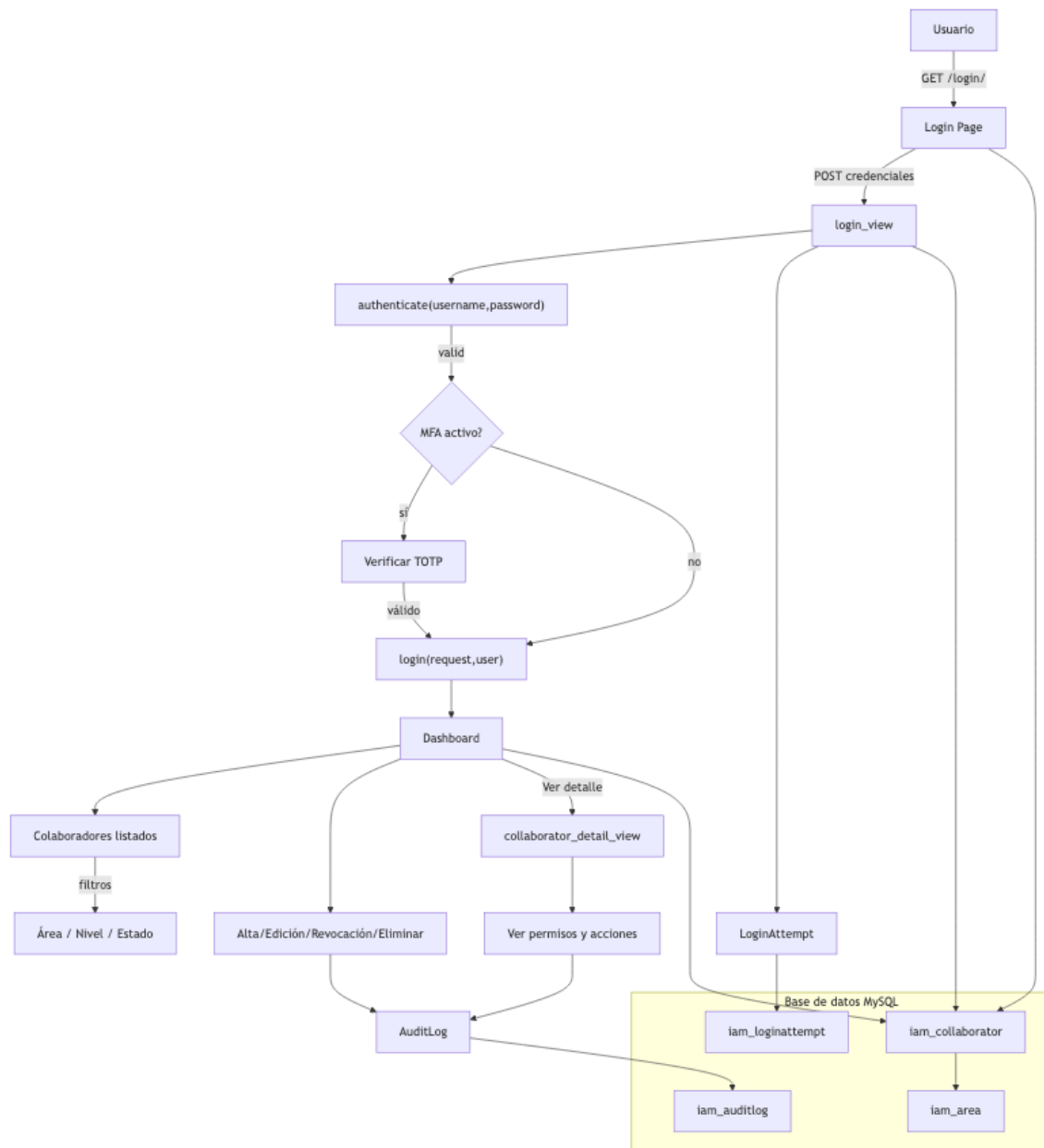
- ``iam/forms.py``

- Formularios de login y gestión de colaboradores.

- ``iam/urls.py``

- Rutas de acceso del panel IAM.

3. Diagrama de funcionamiento



3.1 Descripción general

El flujo principal es:

1. El usuario accede a `/login/`.

2. El formulario se envía a ``login_view``.
3. Django autentica con ``authenticate()``.
4. Si MFA está activo, se valida el token TOTP.
5. Se crea un registro en ``LoginAttempt``.
6. Si el usuario es válido y activo, se redirige al dashboard.
7. El dashboard filtra colaboradores según nivel y área.
8. Las acciones de alta, edición, revocación y eliminación generan entradas en ``AuditLog``.

3.2 Roles y niveles de acceso

- Nivel 1: Administración total.
- Nivel 2: Coordinadores con acceso a su propia área y creación de usuarios de nivel 3/4.
- Nivel 3: Personal operativo dentro de su área.
- Nivel 4: Personal externo con acceso mínimo.

4. Modelo de datos SQL

4.1 Tablas principales

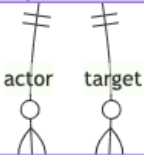
- ``iam_area``
- ``iam_collaborator``
- ``iam_auditlog``
- ``iam_loginattempt``

4.2 Relacionamiento

- ``iam_collaborator.area`` → ``iam_area.id``
- ``iam_collaborator.created_by`` → ``iam_collaborator.id``
- ``iam_collaborator.revoked_by`` → ``iam_collaborator.id``
- ``iam_auditlog.actor`` → ``iam_collaborator.id``
- ``iam_auditlog.target`` → ``iam_collaborator.id``

5. Diagrama SQL de la base de datos

iam_auditlog		
int	id	PK
int	actor_id	FK
int	target_id	FK
varchar	action	
text	details	
datetime	created_at	



iam_collaborator		
int	id	PK
varchar	username	
varchar	email	
varchar	first_name	
varchar	last_name	
varchar	phone	
int	access_level	
varchar	role	
varchar	job_title	
date	onboarding_date	
varchar	internal_id	
bool	is_revoked	
datetime	revoked_at	
int	revoked_by_id	FK
int	created_by_id	FK
datetime	created_at	
bool	is_deleted	
datetime	deleted_at	
varchar	totp_secret	

iam_loginattempt		
int	id	PK
varchar	username	
varchar	ip_address	
bool	successful	
datetime	created_at	

5.1 Campos principales de `iam\_collaborator`

- `id` (PK)
- `username`
- `email`
- `first\_name`
- `last\_name`
- `area\_id`
- `access\_level`
- `role`
- `job\_title`
- `onboarding\_date`
- `internal\_id`
- `is\_revoked`
- `revoked\_at`
- `revoked\_by\_id`
- `created\_by\_id`
- `created\_at`
- `is\_deleted`
- `deleted\_at`
- `totp\_secret`
- `mfa\_enabled`

5.2 Claves de auditoría y seguridad

- `AuditLog` registra quién hizo qué y cuándo.
- `LoginAttempt` registra intentos fallidos para bloqueo temporal.

6. Estado actual de la implementación

- MySQL está configurado como backend.
- La base `casamonarca` existe y tiene tablas migradas.
- El servidor Django responde en `http://127.0.0.1:8000/`.
- El comando `seed\_data` ya generó usuarios de prueba.

7. Recomendaciones de uso

- Usar `admin` / `Admin2026!` para login inicial.
- Ejecutar `python manage.py seed\_data` siempre que se restaure la base de datos nueva.
- Mantener las variables de entorno definidas para MySQL.

8. Consideraciones de seguridad

- El sistema no usa servicios externos de identidad.
- El hashing de contraseña es manejado por Django.
- MFA TOTP es opcional y se gestiona localmente.
- Cambiar las contraseñas de prueba en un entorno real.